

Lab 1:

The screenshot shows a PostgreSQL query editor interface. The top navigation bar includes tabs for Dashboard, Properties, SQL, Statistics, Dependencies, Dependents, Processes, and a dropdown for the current database (dvdrental/postgres). The SQL tab is active, displaying a query:

```
1 EXPLAIN (ANALYZE, BUFFERS)
2 SELECT c.first_name, c.last_name, r.rental_date
3 FROM customer c
4 JOIN rental r ON c.customer_id = r.customer_id
5 WHERE c.customer_id = 1;
```

Below the query editor, the 'Data Output' tab is selected, showing the 'QUERY PLAN' for the executed query. The plan details are as follows:

Step	Operation
1	Nested Loop (cost=0.28..359.16 rows=32 width=21) (actual time=0.132..1.166 rows=32.00 loops=1)
2	Buffers: shared hit=151 read=2
3	-> Index Scan using customer_pkey on customer c (cost=0.28..8.29 rows=1 width=17) (actual time=0.111..0.112 rows=1.00 loops=1)
4	Index Cond: (customer_id = 1)
5	Index Searches: 1
6	Buffers: shared hit=1 read=2
7	-> Seq Scan on rental r (cost=0.00..350.55 rows=32 width=10) (actual time=0.019..1.049 rows=32.00 loops=1)
8	Filter: (customer_id = 1)
9	Rows Removed by Filter: 16012
10	Buffers: shared hit=150
11	Planning:
12	Buffers: shared hit=200
13	Planning Time: 4.152 ms
14	Execution Time: 1.198 ms

At the bottom of the output, a summary bar indicates: Total rows: 14, Query complete 00:00:00.053.

Analysis Questions:

1. Outer table: customer

Inner table: rental

2. customer table returns 1 row

Hash Join will become too expensive because the cost for building a Hash table is far more expensive than just scanning the outer table then scanning the inner table using a Nested Loop.

3. The outer customer table only produced 1 row so the inner table only loops 1 time.

Lab 2:

The screenshot shows a PostgreSQL query editor interface. At the top, there are tabs for Dashboard, Properties, SQL, Statistics, Dependencies, Dependents, and Processes. Below the tabs, a dropdown menu shows 'dvdrental/postgres@PostgreSQL'. A toolbar contains icons for file operations, query execution, and settings. The main area is divided into 'Query' and 'Query History' tabs. The 'Query' tab contains the following SQL code:

```
10 -- JOIN rental r ON c.customer_id = r.customer_id;
11
12 EXPLAIN (ANALYZE, BUFFERS)
13 SELECT f.title, l.name as language
14 FROM film f
15 JOIN language l ON f.language_id = l.language_id;
```

Below the query editor, there are tabs for 'Data Output', 'Messages', and 'Notifications'. The 'Data Output' tab is active, showing a table with 14 rows. The table has two columns: 'QUERY PLAN' and 'text'. The first row is a header, and the subsequent rows show the execution plan for the query. The plan indicates a Hash Join operation with a cost of 1.14..103.62, 1000 rows, and a width of 99. The join condition is (f.language_id = l.language_id). The plan also shows the sequence of operations: Seq Scan on film f, Hash Cond, Buffers: shared hit=89, Seq Scan on language l, Buffers: shared hit=1, and Planning. The total execution time is 0.486 ms.

	QUERY PLAN	text
1	Hash Join (cost=1.14..103.62 rows=1000 width=99) (actual time=0.043..0.426 rows=1000.00 loops=1)	
2	Hash Cond: (f.language_id = l.language_id)	
3	Buffers: shared hit=89	
4	-> Seq Scan on film f (cost=0.00..98.00 rows=1000 width=17) (actual time=0.022..0.259 rows=1000.00 loops=1)	
5	Buffers: shared hit=88	
6	-> Hash (cost=1.06..1.06 rows=6 width=88) (actual time=0.015..0.015 rows=6.00 loops=1)	
7	Buckets: 1024 Batches: 1 Memory Usage: 9kB	
8	Buffers: shared hit=1	
9	-> Seq Scan on language l (cost=0.00..1.06 rows=6 width=88) (actual time=0.010..0.011 rows=6.00 loops=1)	
10	Buffers: shared hit=1	
11	Planning:	
12	Buffers: shared hit=120 dirtied=1	
13	Planning Time: 3.999 ms	
14	Execution Time: 0.486 ms	

Analysis Questions:

1. Table film is being Hashed, table language is being Probed.
2. The Memory Usage is 9kb

3. Because at first it only need 9kB so when we set work_mem = '64kB' it still the same as start.

```
7  EXPLAIN (ANALYZE, BUFFERS)
8  SELECT f.title, l.name as language
9  FROM film f
10 JOIN language l ON f.language_id = l.language_id;
11
12 SET work_mem = '64kB';
```

Data Output Messages Notifications

Showing rows: 1 to 12 Page No: 1

	QUERY PLAN
1	Hash Join (cost=1.14..103.62 rows=1000 width=99) (actual time=0.040..0.415 rows=1000.00 loops=1)
2	Hash Cond: (f.language_id = l.language_id)
3	Buffers: shared hit=89
4	-> Seq Scan on film f (cost=0.00..98.00 rows=1000 width=17) (actual time=0.021..0.241 rows=1000.00 loops=1)
5	Buffers: shared hit=88
6	-> Hash (cost=1.06..1.06 rows=6 width=88) (actual time=0.011..0.011 rows=6.00 loops=1)
7	Buckets: 1024 Batches: 1 Memory Usage: 9kB
8	Buffers: shared hit=1
9	-> Seq Scan on language l (cost=0.00..1.06 rows=6 width=88) (actual time=0.006..0.007 rows=6.00 loops=1)
10	Buffers: shared hit=1
11	Planning Time: 0.191 ms
12	Execution Time: 0.465 ms

Total rows: 12 Query complete 00:00:00.042

Lab 3

Interface showing PostgreSQL query execution results for a query on the 'dvdrental' database.

```

14 SET enable_hashjoin = OFF;
15 SET enable_nestloop = OFF;
16
17 EXPLAIN (ANALYZE, BUFFERS)
18 SELECT f.film_id, f.title, i.inventory_id
19 FROM film f
20 JOIN inventory i ON f.film_id = i.film_id
21 ORDER BY f.film_id;

```

QUERY PLAN

1	Merge Join (cost=412.64..622.36 rows=4581 width=23) (actual time=1.845..3.851 rows=4581.00 loops=1)
2	Merge Cond: (f.film_id = i.film_id)
3	Buffers: shared hit=111 read=1, temp read=10 written=12
4	-> Index Scan using film_pkey on film f (cost=0.28..127.33 rows=1000 width=19) (actual time=0.006..0.823 rows=1000.00 loops=1)
5	Index Searches: 1
6	Buffers: shared hit=83 read=1
7	-> Materialize (cost=412.37..435.27 rows=4581 width=6) (actual time=1.826..2.441 rows=4581.00 loops=1)
8	Storage: Memory Maximum Storage: 17kB
9	Buffers: shared hit=28, temp read=10 written=12
10	-> Sort (cost=412.37..423.82 rows=4581 width=6) (actual time=1.820..2.120 rows=4581.00 loops=1)
11	Sort Key: i.film_id
12	Sort Method: external merge Disk: 80kB
13	Buffers: shared hit=28, temp read=10 written=12
14	-> Seq Scan on inventory i (cost=0.00..70.81 rows=4581 width=6) (actual time=0.019..0.337 rows=4581.00 loops=1)

Total rows: 19 Query complete 00:00:00.063

1. No, it use an Index Scan using film_pkey on film f
2. Yes, it switch back to Hash Join. Because it doesn't need to sort anymore, it more effiecient to hash the film table which is small enough to fit in work_mem, then the database only needs to perform a seq scan on inventory table.

Query

Query History

16

17

18

19

20

21

RESET ALL;

EXPLAIN (ANALYZE, BUFFERS)

SELECT f.film_id, f.title, i.inventory_id

FROM film f

JOIN inventory i ON f.film_id = i.film_id;

Data Output

Messages

Notifications

≡+

📄

▼

📋

▼

🗑️

🗄️

⬇️

📈

SQL

Showing rows: 1 to 14

✎

QUERY PLAN

text

🔒

1

Hash Join (cost=110.50..193.39 rows=4581 width=23) (actual time=0.517..1.248 rows=4581.00 loops=1)

2

Hash Cond: (i.film_id = f.film_id)

3

Buffers: shared hit=113

4

-> Seq Scan on inventory i (cost=0.00..70.81 rows=4581 width=6) (actual time=0.023..0.168 rows=4581.00 loo...

5

Buffers: shared hit=25

6

-> Hash (cost=98.00..98.00 rows=1000 width=19) (actual time=0.458..0.458 rows=1000.00 loops=1)

7

Buckets: 1024 Batches: 1 Memory Usage: 59kB

8

Buffers: shared hit=88

9

-> Seq Scan on film f (cost=0.00..98.00 rows=1000 width=19) (actual time=0.015..0.365 rows=1000.00 loo...

10

Buffers: shared hit=88

11

Planning:

12

Buffers: shared hit=6

13

Planning Time: 0.330 ms

14

Execution Time: 1.380 ms

Hands-on lab

1. The database system calculates that Seqscan is more efficient than Index scan so in both case the cost is the same. The time of the second run is faster because it already have the data loaded in RAM from the previous scan.

2.

28	
29	EXPLAIN ANALYZE
30	SELECT c.first_name, c.last_name, r.rental_date, i.film_id
31	FROM customer c
32	JOIN rental r ON c.customer_id = r.customer_id
33	JOIN inventory i ON r.inventory_id = i.inventory_id;
34	
Data Output Messages Notifications	
≡ + 📄 ▼ 📋 ▼ 🗑️ 🗄️ ⬇️ 📈 SQL Showing rows: 1 to 22	
	QUERY PLAN
	text
1	Hash Join (cost=150.55..545.57 rows=16044 width=23) (actual time=1.228..6.652 rows=16044.00 loops=1)
2	Hash Cond: (r.inventory_id = i.inventory_id)
3	Buffers: shared hit=184
4	-> Hash Join (cost=22.48..375.33 rows=16044 width=25) (actual time=0.195..3.312 rows=16044.00 loops=1)
5	Hash Cond: (r.customer_id = c.customer_id)
6	Buffers: shared hit=159
7	-> Seq Scan on rental r (cost=0.00..310.44 rows=16044 width=14) (actual time=0.018..0.771 rows=16044.00 loop...
8	Buffers: shared hit=150
9	-> Hash (cost=14.99..14.99 rows=599 width=17) (actual time=0.167..0.169 rows=599.00 loops=1)
10	Buckets: 1024 Batches: 1 Memory Usage: 39kB
11	Buffers: shared hit=9
12	-> Seq Scan on customer c (cost=0.00..14.99 rows=599 width=17) (actual time=0.024..0.102 rows=599.00 loo...
13	Buffers: shared hit=9
14	-> Hash (cost=70.81..70.81 rows=4581 width=6) (actual time=1.011..1.011 rows=4581.00 loops=1)
15	Buckets: 8192 Batches: 1 Memory Usage: 243kB
Total rows: 22 Query complete 00:00:00.044	

The database system chose Hash Join

29	ANALYZE customer, rental, inventory;
30	EXPLAIN ANALYZE
31	SELECT c.first_name, c.last_name, r.rental_date, i.film_id
32	FROM customer c
33	JOIN rental r ON c.customer_id = r.customer_id
34	JOIN inventory i ON r.inventory_id = i.inventory_id;

Data Output

Messages

Notifications

≡

📄

▼

📋

▼

🗑

🗄

⬇

📈

SQL

Showing rows: 1 to 22

✎

QUERY PLAN

text

🔒

1

Hash Join (cost=150.55..545.57 rows=16044 width=23) (actual time=0.875..5.450 rows=16044.00 loops=1)

2

Hash Cond: (r.inventory_id = i.inventory_id)

3

Buffers: shared hit=184

4

-> Hash Join (cost=22.48..375.33 rows=16044 width=25) (actual time=0.169..2.913 rows=16044.00 loops=1)

5

Hash Cond: (r.customer_id = c.customer_id)

6

Buffers: shared hit=159

7

-> Seq Scan on rental r (cost=0.00..310.44 rows=16044 width=14) (actual time=0.011..0.603 rows=16044.00 loop...

8

Buffers: shared hit=150

9

-> Hash (cost=14.99..14.99 rows=599 width=17) (actual time=0.149..0.150 rows=599.00 loops=1)

10

Buckets: 1024 Batches: 1 Memory Usage: 39kB

11

Buffers: shared hit=9

12

-> Seq Scan on customer c (cost=0.00..14.99 rows=599 width=17) (actual time=0.010..0.081 rows=599.00 loo...

13

Buffers: shared hit=9

14

-> Hash (cost=70.81..70.81 rows=4581 width=6) (actual time=0.684..0.684 rows=4581.00 loops=1)

15

Buckets: 8192 Batches: 1 Memory Usage: 243kB

Total rows: 22

Query complete 00:00:00.048

Plan isn't change because the statistics of the database is corected before analyze.